ENGINEERING REFERENCE

vdrapkinTimerManager

Windows RTOS-Style Timer Manager

Architecture, Protocol, Implementation, Sample Application, and Verification

---

A console-based C reference implementation of a client/server timer service, including its public interface, communication protocol, scheduling algorithm, sample application, and automated tests.

Prepared by Vitaly Drapkin

Version 1.6 | October 25, 2026

Document Purpose

This manual describes the complete C vdrapkinTimerManager project. It introduces the externally visible timer behavior first, then defines the participating components, communication protocol, scheduling algorithm, server processing, sample application, and synchronization rules. The final sections describe the build procedure and the automated tests that verify normal operation, rejected requests, capacity limits, concurrency, and arithmetic boundaries.

Contents

1. Overview 5
2. Scope and Design Principles 6
2.1 Included 6
2.2 Deliberate Exclusions 6
2.3 Coding Contract 7
3. Public Client API 7
3.1 ACK Status Values 8
4. System Architecture 8
4.1 Ownership Boundaries 9
5. Shared Binary Protocol 9
5.1 Message Catalog 11
5.2 Client-to-Server Messages 12
5.3 Server-to-Client Messages 13
5.4 Validation Contract 14
6. Timer Core 15
6.1 Active and Inactive Lists 16
6.2 Delta Representation 16
6.3 Expiry and Overflow 17

Copyright (c) 2026 Vitaly Drapkin

7. Server Implementation 17
7.1 Startup 18
7.2 Wait-Set Construction 18
7.3 Listener and Capacity Handling 19
7.4 Client Commands 19
7.5 Expiry Callback 19
7.6 Disconnect and Shutdown 19
8. Sample Connection State Machine 20
9. Concurrency and Synchronization 21
9.1 Client API Thread Safety 21
9.2 Global State Access Rule 22
10. Error Handling and Defensive Behavior 23
11. Building and Running 23
11.1 Prerequisites 23
11.2 Configure and Build 24
11.3 Run 24
11.4 Run Tests 24
12. Verification Strategy 24
12.1 Why Negative Tests Matter 25
12.2 Complete Automated Test Catalog 25
13. Current Limitations and Future Work 30
14. AI-Assisted Development Workflow 31
Appendix A. Project Layout 32
Appendix B. Key Constants 33
Appendix C. Design Invariants 33
Appendix D. Representative Execution Traces 35
D.1 Sample Application State Machine Run 35
D.2 Server StartTimer Adjustment and Expiry Delivery 37
D.3 Automated Regression Result 38

Copyright (c) 2026 Vitaly Drapkin

Appendix E. StartTimer API Timestamp Revision 38

Copyright (c) 2026 Vitaly Drapkin

1. Overview

vdrapkinTimerManager provides an RTOS-style timer service for Windows applications. In this manual, an application is a client program that uses the service, and a logical timer is a service-managed object identified by a numeric timer ID. Each logical timer is one-shot: one successful start can produce at most one expiry, and the service does not restart the timer automatically.

The public C API defines the timer lifecycle. CreateTimer creates a logical timer and returns its timer ID without starting a countdown. StartTimer starts a countdown using a duration in milliseconds. If the application calls StartTimer again before that timer expires, the service cancels the current countdown and starts a new countdown from the time of the second call using the new duration. StopTimer cancels a running countdown without deleting the timer, and DeleteTimer releases the timer ID and its associated resources.

CreateTimer, StartTimer, StopTimer, and DeleteTimer are synchronous commands: each function waits until the service returns an acknowledgment (ACK) containing the command result. Expiry delivery is asynchronous. The API provides a notification event, which is a Windows wait handle that becomes signaled when at least one expired timer ID is available. After the event is signaled, the application calls GetExpiredTimer until the first-in, first-out (FIFO) expiry queue is empty.

The API implementation is a client library linked with the application. It communicates with the vdrapkinTimerManager console server through \\.\pipe\vdrapkinTimerManager, a Windows named pipe used as the interprocess communication channel. The server supports at most 61 simultaneous client connections, and the public API may be called by multiple application threads. Figure 1 shows these externally visible components. Later sections define the protocol, scheduling data structures, Windows wait objects, and synchronization used to implement this behavior.

Copyright (c) 2026 Vitaly Drapkin

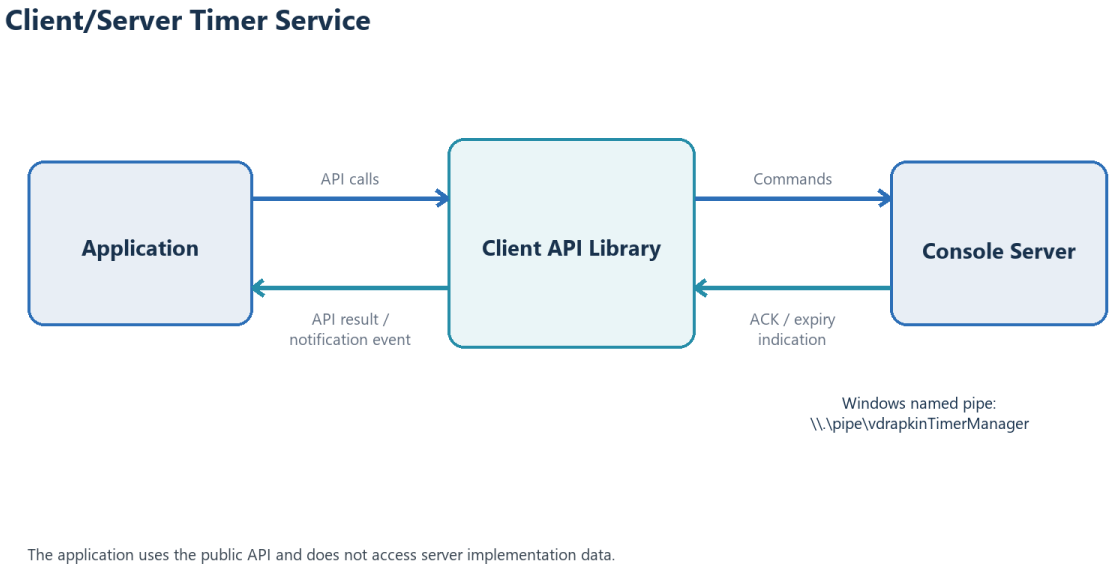


Figure 1. Client/server timer-service overview

2. Scope and Design Principles

The project is intentionally small enough to study from end to end, yet complete enough to exercise timer ownership, asynchronous delivery, cleanup, and failure handling. Its design choices favor visible control flow and deterministic behavior over production-scale throughput.

2.1 Included

The implemented lifecycle covers creation, start or restart, stop, expiry, and deletion. Each timer is associated with the client connection that created it, so another client cannot operate on that timer and disconnect cleanup can release all timers associated with the lost connection.

The project includes asynchronous client/server communication, event-based expiry notification, orderly Ctrl+C shutdown, and a sample connection state machine driven by timer expiries. Unit, integration, robustness, capacity, concurrency, and sample-specific tests verify the documented behavior.

2.2 Deliberate Exclusions

The example supports one-shot timers only. It does not add periodic scheduling, callback-based application delivery, persistent timer storage, or automatic server startup because those features would obscure the request, wait, and expiry paths that the sample is meant to expose.

Copyright (c) 2026 Vitaly Drapkin

The server also avoids I/O completion port (IOCP) worker threads, a separate scheduler thread, Windows service hosting, authentication, authorization, and cross-version negotiation. Timer allocation is limited by process memory rather than a configured timer count, and allocation failure is reported through the public API as an internal error. These boundaries keep the implementation educational rather than presenting it as a commercially hardened service.

2.3 Coding Contract

Each function validates all parameters before entering its operational logic. A programmer-contract violation asserts in a Debug build when execution cannot continue safely, while failures that a caller can handle are returned through the appropriate status value.

3. Public Client API

The public API converts the lifecycle described in Section 1 into six callable functions. Four command functions create, start, stop, or delete a timer and return an ACK status. Two notification functions provide the expiry event and retrieve queued timer IDs. Table 1 specifies the exact function names, returned information, and caller obligations.

Table 1. Public client API

API	Behavior	Important rule
vdrTimerManager_CreateTimer	Creates one service-managed logical timer without starting its countdown and returns its nonzero ID.	This must be the first public call because the library performs its initialization automatically during this call.
vdrTimerManager_StartTimer	Starts a countdown for a timer that is not running or restarts a running timer with a new duration. The API captures api_start_timestamp_ms before sending the StartTimer command so the server can account for Windows scheduling delay before dispatch.	The duration_ms argument must be nonzero, and the caller waits for the server ACK. If the adjusted duration has already elapsed by the time the server processes the command, the server returns ACK OK first and then sends the asynchronous expiry indication.
vdrTimerManager_StopTimer	Cancels the running countdown without deleting the logical timer.	Stopping a timer that is not running succeeds and has no additional effect, making the operation idempotent.
vdrTimerManager_DeleteTimer	Deletes the logical timer.	A successful delete makes the timer ID

Copyright (c) 2026 Vitaly Drapkin

API	Behavior	Important rule
		invalid for later commands.
vdrTimerManager_GetNotificationEvent	Returns the API-owned notification event described in Section 1.	The application may wait on the handle but must not close it.
vdrTimerManager_GetExpiredTimer	Removes one expired timer ID from the FIFO queue.	The API resets the event after the final queued ID is removed.

3.1 ACK Status Values

Each command returns one vdrAckStatus_type value to report success or the reason for failure. When a command concerns an existing timer, the response also identifies that timer. If timer creation fails before an ID is allocated, the API returns TIMER_ID_INVALID, whose reserved numeric value is zero. Table 2 defines every ACK status and its numeric value.

Table 2. ACK status values

Status	Value	Meaning
OK	0	The command completed successfully.
INVALID_TIMER_ID	1	The timer ID is zero or does not exist.
INVALID_DURATION	2	The requested start duration is zero.
TIMER_NOT_OWNED	3	The timer exists but belongs to another client pipe.
SERVER_BUSY	4	The value is reserved for future capacity or back-pressure behavior.
PROTOCOL_ERROR	5	The message header, type, byte count, or payload metadata is invalid.
INTERNAL_ERROR	6	Allocation, a Windows API, or an unexpected internal operation failed.
SERVER_NOT_AVAILABLE	7	The client cannot establish or continue pipe communication.
NO_EXPIRED_TIMERS	8	The client expiry queue is currently empty.

4. System Architecture

Timer processing is divided among four components. The application calls the public functions defined in Section 3. The client API library converts each command call into a request and converts each response into an API result. The console server validates requests and

Copyright (c) 2026 Vitaly Drapkin

coordinates communication with scheduling. The timer core is the server module that calculates when each logical timer expires. Table 3 assigns each responsibility to one component so that ownership is not duplicated.

Table 3. System components and responsibilities

Component	Primary responsibility	Concurrency
Application	Calls the public API, assigns meaning to each timer, and combines the expiry event with its own wait handles.	The application chooses its threading model. The sample keeps state-machine logic on one main thread.
Client API	Converts public function calls into protocol requests and converts server responses into command results or queued expiry IDs.	The library serializes command exchanges and manages asynchronous expiry delivery. Section 9.1 defines the internal thread and lock behavior.
Console server	Owns pipes, validates protocol messages, dispatches timer operations, and sends ACK or expiry messages.	One main thread performs request and timer work. The console handler only signals shutdown.
Timer core	Stores internal scheduling state and calculates the next logical timer expiry.	The core has no internal lock because the server serializes every call.

4.1 Ownership Boundaries

The server owns the internal timer object created for each successful CreateTimer command and associates that object with the client connection that requested it. The timer core may update the object's scheduling state, but only the server communicates the command result or expiry indication to the client.

The client API library owns the connection and the notification event returned to the application. The application may include that event in a wait operation, but it must not close the handle. The library also owns all internal resources used to match command responses and retain timer IDs until the application retrieves them.

5. Shared Binary Protocol

The named-pipe protocol uses fixed-size C structures with compiler-native alignment. The client and server are built from the same protocol header and compiler application binary interface

Copyright (c) 2026 Vitaly Drapkin

(ABI), so both executables use the same field offsets and structure sizes. A layout change therefore requires both executables to be rebuilt together. Every request, ACK, and expiry indication begins with vdrMessageHeader_type, the common header structure listed in Table 4. The request union and response union each place that header first so the receiver can validate it before accessing message-specific fields.

Copyright (c) 2026 Vitaly Drapkin

Table 4. Message header fields

Header field	Type	Meaning
magic	uint32_t	Contains 0x544D4752 ('TMGR') so the receiver can reject unrelated bytes.
version	uint16_t	Contains protocol version, so incompatible endpoints fail before payload access.
message_type	uint16_t	Contains a vdrMessageType_type numeric value that selects the valid message structure.
request_id	uint32_t	Correlates one command with its ACK. Asynchronous expiry events use zero.
payload_size	uint32_t	Reports the bytes following the common 16-byte header for exact-size validation.

```
typedef struct vdrMessageHeader_tag {
    uint32_t magic;
    uint16_t version;
    uint16_t message_type;
    uint32_t request_id;
    uint32_t payload_size;
} vdrMessageHeader_type;
```

5.1 Message Catalog

After validating the common header, the receiver uses message_type to select one of six values from vdrMessageType_type, the enumeration that identifies every protocol message. The numeric values are part of the shared protocol contract, and the received byte count must match the fixed structure size associated with the selected type. Table 5 lists the mapping among symbolic message names, numeric values, structure sizes, and payload fields.

Table 5. Protocol message catalog

Direction	Message	Value	Bytes	Payload
Client to server	CREATE_TIMER_REQ	1	16	Contains only the common header.
Client to server	START_TIMER_REQ	2	40	Contains timer_id, duration_ms, and api_start_timestamp_ms.
Client to server	STOP_TIMER_REQ	3	24	Contains timer_id.
Client to server	DELETE_TIMER_REQ	4	24	Contains timer_id.
Server to client	ACK_RSP	5	32	Contains status, reserved, and timer_id.

Copyright (c) 2026 Vitaly Drapkin

Direction	Message	Value	Bytes	Payload
Server to client	TIMER_EXPIRED_EVT	6	24	Contains the expired timer_id.

5.2 Client-to-Server Messages

The client API represents every command with one member of vdrClientToServerMessage_type. Each request structure begins with vdrMessageHeader_type. This common prefix lets the server validate the protocol identity, version, request ID, message type, and payload size before it reads command-specific fields. After validation, message_type identifies the only union member that may be accessed.

CreateTimer request. The application uses CreateTimer to ask the server to allocate a logical timer owned by the requesting client connection. The request needs no command-specific payload, so the structure contains only the common header. The client API sets message_type to vdrMESSAGE_TYPE_CREATE_TIMER_REQ and payload_size to zero. A successful ACK returns the newly allocated timer ID; if allocation fails, the ACK carries TIMER_ID_INVALID.

```
typedef struct vdrCreateTimerReq_tag {
    vdrMessageHeader_type header;
} vdrCreateTimerReq_type;
```

StartTimer request. The application uses StartTimer to start an inactive timer or restart an active timer with a new one-shot duration. timer_id selects the timer owned by the client, and duration_ms supplies the requested duration in milliseconds. api_start_timestamp_ms is a monotonic timestamp captured by the client API immediately before transmission. The server compares that timestamp with its current monotonic time and subtracts transport and scheduling delay before it schedules the timer. message_type is vdrMESSAGE_TYPE_START_TIMER_REQ.

```
typedef struct vdrStartTimerReq_tag {
    vdrMessageHeader_type header;
    TimerId_type timer_id;
    uint64_t duration_ms;
    uint64_t api_start_timestamp_ms;
} vdrStartTimerReq_type;
```

Copyright (c) 2026 Vitaly Drapkin

StopTimer request. The application uses StopTimer to stop the countdown identified by timer_id without deleting the timer. The server removes an active timer from the scheduling list and returns it to the inactive list; stopping an already inactive timer is also successful. message_type is vdrMESSAGE_TYPE_STOP_TIMER_REQ, and the timer remains available for a later StartTimer request.

```
typedef struct vdrStopTimerReq_tag {
    vdrMessageHeader_type header;
    TimerId_type timer_id;
} vdrStopTimerReq_type;
```

DeleteTimer request. The application uses DeleteTimer to release the logical timer identified by timer_id. The server verifies ownership, removes the timer from either the active or inactive list, and frees its allocation. An active timer that is deleted cannot later produce an expiry indication. message_type is vdrMESSAGE_TYPE_DELETE_TIMER_REQ.

```
typedef struct vdrDeleteTimerReq_tag {
    vdrMessageHeader_type header;
    TimerId_type timer_id;
} vdrDeleteTimerReq_type;
```

Client-to-server union. The client API and server use vdrClientToServerMessage_type as the common storage type for command transmission and reception. The union does not contain several commands in one pipe message; it provides one buffer large enough for the largest request and several typed views of the same bytes. header is the first union member so the server can inspect the common prefix before selecting create_timer_req, start_timer_req, stop_timer_req, or delete_timer_req according to message_type.

```
typedef union vdrClientToServerMessage_tag {
    vdrMessageHeader_type header;
    vdrCreateTimerReq_type create_timer_req;
    vdrStartTimerReq_type start_timer_req;
    vdrStopTimerReq_type stop_timer_req;
    vdrDeleteTimerReq_type delete_timer_req;
} vdrClientToServerMessage_type;
```

5.3 Server-to-Client Messages

The server represents every acknowledgement or asynchronous indication with one member of vdrServerToClientMessage_type. Each concrete structure again begins with vdrMessageHeader_type. The client receiver thread validates the common prefix and then uses message_type to decide whether the received bytes contain an ACK response or a timer-expiry event.

Copyright (c) 2026 Vitaly Drapkin

ACK response. The server sends one vdrAckRsp_type for every accepted command message, whether the command succeeds or fails. The header request_id echoes the identifier assigned by the client API, allowing the waiting API call to match the ACK to its command. ack_status carries the numeric vdrAckStatus_type result, reserved must be zero, and timer_id identifies the relevant timer when one is available. When no valid timer ID exists, such as after failed timer creation, timer_id is TIMER_ID_INVALID. message_type is vdrMESSAGE_TYPE_ACK_RSP.

```
typedef struct vdrAckRsp_tag {
    vdrMessageHeader_type header;
    uint32_t ack_status;
    uint32_t reserved;
    TimerId_type timer_id;
} vdrAckRsp_type;
```

Timer-expiry event. The server sends vdrTimerExpiredEvt_type when a one-shot timer reaches its scheduled expiry. Before sending the event, the server moves the timer from the active list to the inactive list so the application may start it again. Because the event is asynchronous and does not complete a pending command, header.request_id is zero. timer_id identifies the expired timer, and message_type is vdrMESSAGE_TYPE_TIMER_EXPIRED_EVT.

```
typedef struct vdrTimerExpiredEvt_tag {
    vdrMessageHeader_type header;
    TimerId_type timer_id;
} vdrTimerExpiredEvt_type;
```

Server-to-client union. The server and client receiver thread use vdrServerToClientMessage_type as common storage for both outbound message forms. As with the command union, its members are alternative typed views of one received message rather than simultaneous contents. The header member exposes the common prefix first; after validation, the receiver reads ack_rsp for vdrMESSAGE_TYPE_ACK_RSP or timer_expired_evt for vdrMESSAGE_TYPE_TIMER_EXPIRED_EVT.

```
typedef union vdrServerToClientMessage_tag {
    vdrMessageHeader_type header;
    vdrAckRsp_type ack_rsp;
    vdrTimerExpiredEvt_type timer_expired_evt;
} vdrServerToClientMessage_type;
```

5.4 Validation Contract

The server validates each message in a fixed sequence. It first requires a complete header, then checks the magic value and protocol version, and then verifies that message_type identifies a supported client command. After identifying the command type, the server requires the exact structure size and verifies that payload_size equals the number of bytes following the common header.

Copyright (c) 2026 Vitaly Drapkin

Command logic reads timer IDs or durations only after all header and size checks succeed. For a malformed request, the server returns an ACK whose status is PROTOCOL_ERROR, the protocol-validation status defined in Table 2, and whose timer ID is the reserved TIMER_ID_INVALID value. A protocol validation failure rejects only the current request and does not close the client connection, so the client may send a corrected request afterward. This validation order prevents untrusted bytes from selecting the wrong union member and verifies that rejecting one message does not make the connection unusable.

MALFORMED MESSAGE BEHAVIOR: For a malformed request, the server returns PROTOCOL_ERROR with TIMER_ID_INVALID before executing payload logic. The server keeps the same client connection open because the error applies to the request rather than to the connection. The robustness test then sends a valid command through that connection to verify continued operation.

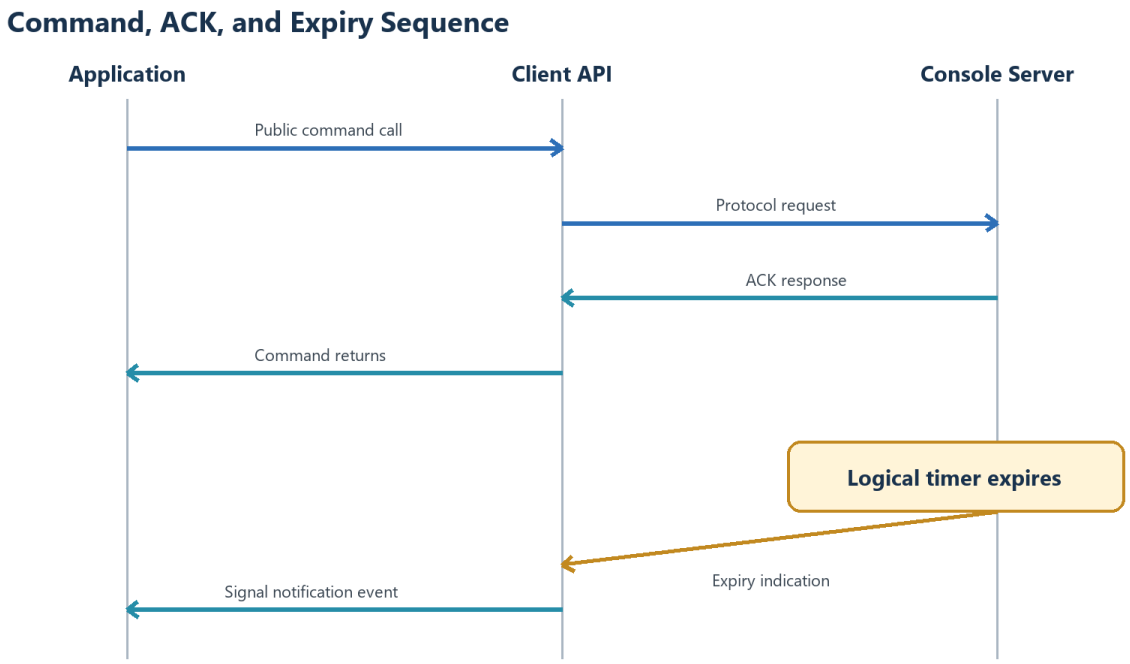


Figure 2. Synchronous command/ACK and asynchronous expiry delivery

6. Timer Core

The timer core implements the scheduling behavior after the API and protocol have accepted a command. For each logical timer, the core allocates one Timer_type structure containing the timer ID, owner connection, duration, timing values, and the next_timer link pointer used to place the structure in a singly linked timer list. A created, stopped, or expired timer is stored on the inactive list, which contains timers without a running countdown. A running timer is stored

Copyright (c) 2026 Vitaly Drapkin

on the active list, which the core orders by expiry time. Instead of allocating one Windows timer per logical timer, the server programs one Windows waitable timer, a kernel wait object that becomes signaled at a specified time, for the next logical expiry. The server supplies the current time and an optional expiry callback, a server function that the core calls after a timer expires.

Within Timer_type, TimerId_type is the project's unsigned 64-bit timer-ID type, and server_pipe is the Windows HANDLE for the owning client connection. The duration_ms field stores the requested duration, and the system_start_time_ms field stores the timestamp supplied when the timer was started. The timer's scheduled expiry is system_start_time_ms plus duration_ms. For the first timer in the active list, time_remainder_ms stores the remaining delay calculated as max(scheduled expiry - now_ms, 0), where now_ms is the current timestamp supplied when the core recalculates the list. It is not another copy of the timer's start timestamp. For every later active timer, time_remainder_ms stores the difference between that timer's scheduled expiry and the preceding timer's scheduled expiry. The next_timer link pointer identifies the next structure in the active or inactive singly linked list.

```
typedef struct Timer_tag {
    TimerId_type id;
    HANDLE server_pipe;
    uint64_t duration_ms;
    uint64_t system_start_time_ms;
    uint64_t time_remainder_ms;
    struct Timer_tag* next_timer;
} Timer_type;
```

6.1 Active and Inactive Lists

A timer begins in the inactive list immediately after creation. Start removes it from the inactive list and inserts it into the active list according to its scheduled expiry, which is the start time plus the requested duration. Starting an already active timer removes the existing active-list entry and reinserts the same allocation using the new start time and duration.

Stop and expiry return the allocation to inactive state, which allows the application to start the timer again. Delete removes the timer from whichever list owns it and frees the allocation. When a pipe disconnects, the core applies that same deletion rule to every timer whose owner matches the lost pipe, leaving timers owned by other clients untouched.

6.2 Delta Representation

The active list is ordered by absolute scheduled expiry, but each node stores a relative delay. Whenever the core recalculates the list, it receives now_ms as the current server timestamp. For the first active timer, the core sets time_remainder_ms to max(scheduled expiry - now_ms,

Copyright (c) 2026 Vitaly Drapkin

0), which is the remaining time until that timer expires at the moment of recalculation. For every later timer, the core sets time_remainder_ms to the difference between that timer's scheduled expiry and the preceding timer's scheduled expiry. Timers with the same scheduled expiry therefore store a zero delta, so one waitable-timer signal causes the server to process every timer due at that instant.

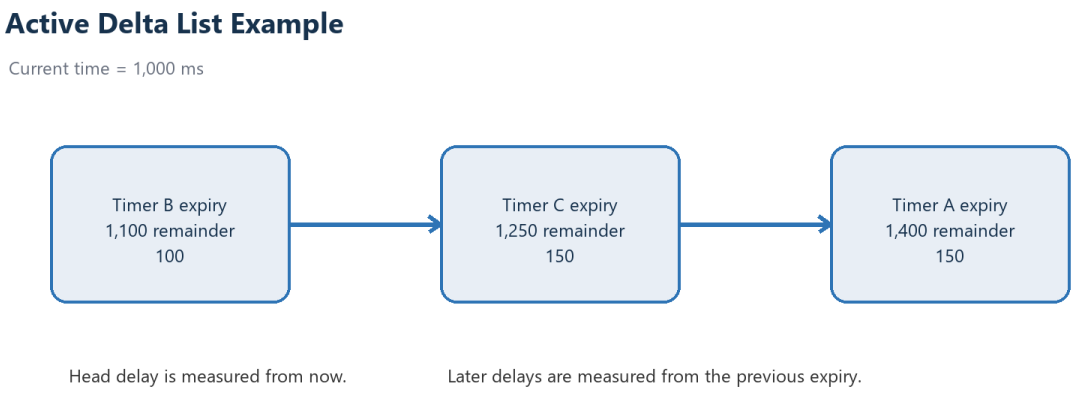


Figure 3. Scheduled-expiry ordering and delta values

6.3 Expiry and Overflow

When the waitable timer signals, vdrTimerCore_ProcessExpired repeatedly removes the first active-list entry whose scheduled expiry has arrived and moves its allocation to the inactive list. The optional server callback runs only after the lists are consistent, so an expiry indication is not sent while the timer still appears active. Scheduled expiry uses saturating uint64_t addition, which clamps start time plus duration to UINT64_MAX instead of wrapping into the past. Boundary tests verify both saturation and ordering near the maximum timestamp.

7. Server Implementation

The server performs request dispatch and timer processing on one main thread. That thread owns the connected-client collection, timer core, waitable timer, and shutdown event. The shutdown event is a manual-reset Windows event that the console handler signals when Ctrl+C occurs. Sequential dispatch prevents simultaneous updates to timer-core state. Section 9 defines how these process-lifetime objects are grouped and accessed.

Named-pipe connection and read operations use Windows overlapped I/O, which allows an operation to remain pending while the main thread waits for other activity. Each pending operation has a completion event. The main thread passes those events, the waitable timer,

Copyright (c) 2026 Vitaly Drapkin

and the shutdown event to WaitForMultipleObjects, a Windows function that blocks until one handle in an array becomes signaled. Because the main thread dispatches all three event classes sequentially, the design does not require a separate scheduler thread. Writes may wait for completion because traceable control flow has priority over throughput in this RTOS simulation. A production server could instead use fully overlapped writes.

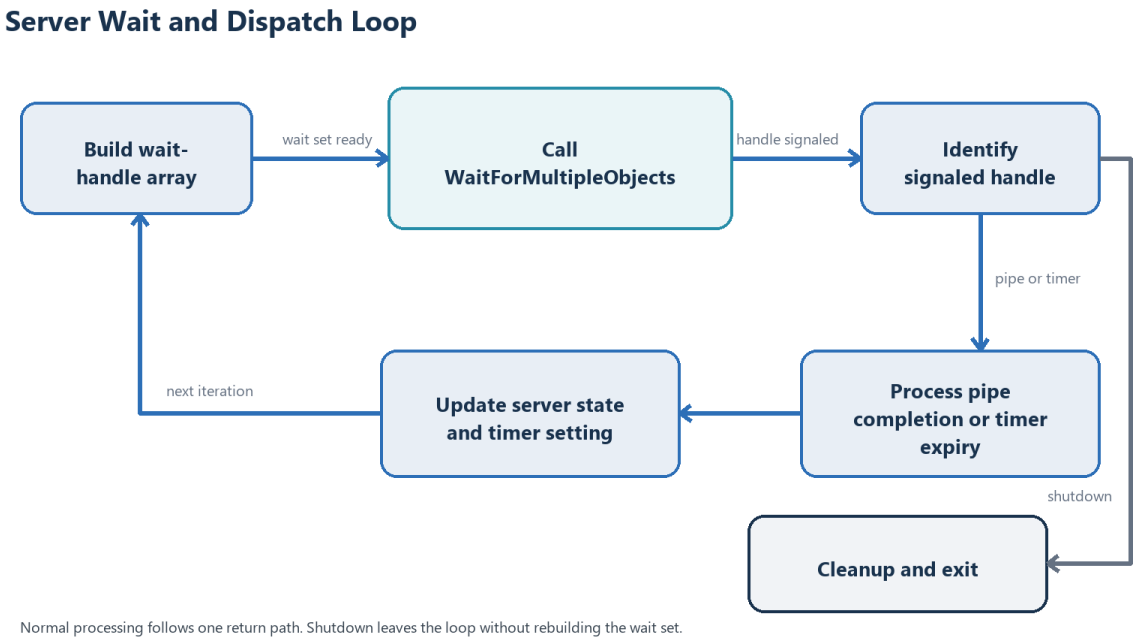


Figure 4. Main server wait-set construction and dispatch

7.1 Startup

Startup creates dependencies in the reverse order from shutdown cleanup. The server first initializes the timer core, then creates the waitable timer and manual-reset shutdown event. After those handles exist, it installs the console control handler and creates the initial listener, which is the named-pipe instance reserved for accepting the next client connection. The server enters its main loop and prints the readiness trace only after the listener's overlapped connection request is represented in the wait set.

7.2 Wait-Set Construction

At the beginning of each loop iteration, the server copies every live pipe event into a local handle array. It then asks the timer core whether an active head exists. When one does, the waitable timer handle for that head is appended to the array. The shutdown event is always appended last. Capturing the timer and shutdown indexes during construction lets the returned wait index identify a pipe, an expiry, or shutdown without maintaining a second dispatch table.

Copyright (c) 2026 Vitaly Drapkin

7.3 Listener and Capacity Handling

When a listener accepts a connection, that same pipe instance becomes a client and is armed for one asynchronous read. The server then creates a replacement listener so another connection can arrive while existing clients remain active. At 61 connected clients, the reserved listener occupies the final pipe slot. An excess connection is disconnected and the listener is rearmed instead of being promoted. If a logical timer is active at that moment, 62 pipe events plus the waitable timer and shutdown event consume exactly 64 handles.

7.4 Client Commands

A client completion event identifies exactly one finished read. The server validates the message header and size before dispatching the selected timer-core operation, then sends a generic ACK containing the resulting status and timer ID. For StartTimer, the server also compares server_now_ms with api_start_timestamp_ms, calculates elapsed_ms, and schedules only the remaining adjusted_duration_ms. If the adjusted duration is zero, the server still sends ACK OK first and then reports the expiry asynchronously, preserving the command/ACK contract while preventing Windows scheduling delay from extending the timer. Because start, stop, delete, or disconnect may change the first timer in the active list, the server recalculates the waitable timer before posting the client's next read. Each client has one Windows OVERLAPPED control structure and one inbound buffer, so allowing only one outstanding asynchronous read per client keeps completion ownership unambiguous.

7.5 Expiry Callback

For each due timer, the core moves the existing Timer_type allocation from active to inactive before invoking the server callback with the timer ID and owner pipe. The callback locates the corresponding client and sends TIMER_EXPIRED_EVT immediately. This handoff avoids a temporary expiry list and ensures that an application may restart the timer as soon as it receives the indication.

7.6 Disconnect and Shutdown

A failed read or send is treated as a disconnect. Removing the pipe first deletes every active and inactive timer owned by that client, compacts the pipe array, and rearms the waitable timer if the active head changed. When Ctrl+C occurs, the console handler signals the shutdown event instead of performing cleanup. WaitForMultipleObjects then returns the shutdown-event index, and the main thread closes every pipe, deinitializes the timer core, cancels and closes the waitable timer, closes the shutdown event, and exits.

Copyright (c) 2026 Vitaly Drapkin

8. Sample Connection State Machine

The sample uses the API-owned notification handle to drive a simulated connection protocol. Its vdrSampleConnectionState_type enumeration defines IDLE = 0, WAITING_FOR_CONNECTION_ESTABLISHMENT = 1, and CONNECTED = 2. CONNECTION_* messages have no payload. Colored trace output identifies each simulated transmission, reception, timer expiry, and state transition while the state machine repeats its operating cycle.

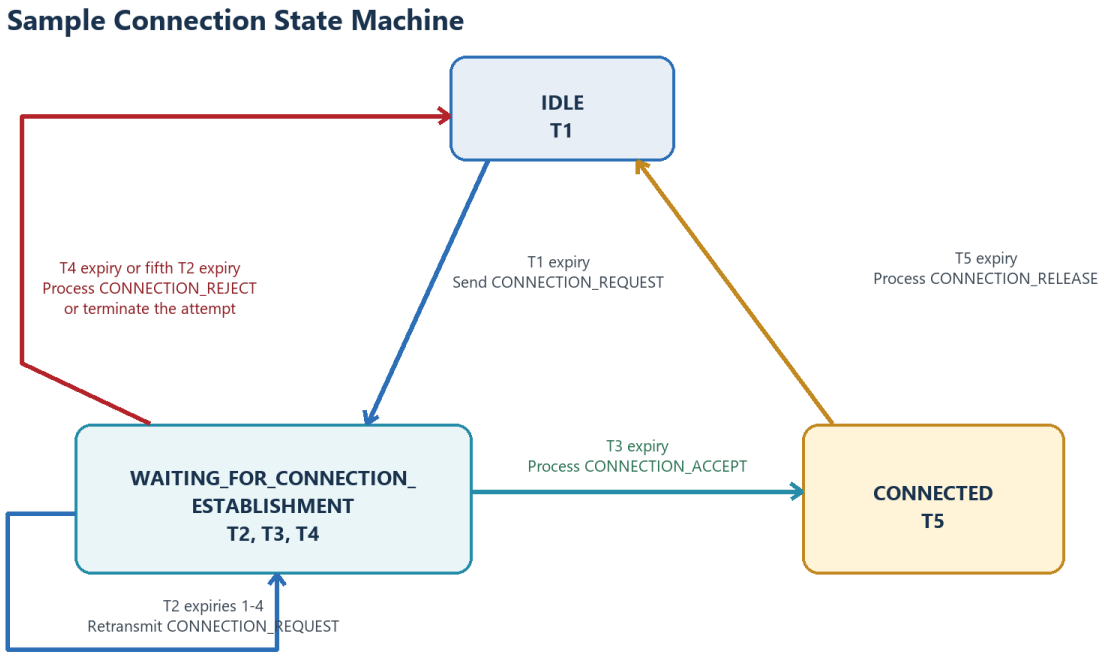


Figure 5. Timer-driven sample application state machine

Table 6. Sample application timers

Timer	Random duration	Meaning when the timer expires
T1	3 to 10 seconds	The application transmits CONNECTION_REQUEST, changes the state from IDLE to WAITING_FOR_CONNECTION_ESTABLISHMENT, and starts the connection-establishment timers.
T2	3 to 5 seconds	The application retransmits CONNECTION_REQUEST. After the fifth T2 expiry, the application terminates the attempt, changes the state to IDLE, and restarts T1.
T3	1 to 24 seconds	The application processes CONNECTION_ACCEPT, stops T4 and T2, changes the state to CONNECTED, and starts T5.
T4	1 to 24 seconds	The application processes CONNECTION_REJECT, stops T3 and T2, changes the state to IDLE, and restarts T1.
T5	5 to 15 seconds	The application processes CONNECTION_RELEASE, changes the state to IDLE, and

Copyright (c) 2026 Vitaly Drapkin

Timer	Random duration	Meaning when the timer expires
		begins the next cycle by starting T1.

In IDLE, the application starts T1 and remains in that state until T1 expires. It then transmits CONNECTION_REQUEST, changes the state to WAITING_FOR_CONNECTION_ESTABLISHMENT, and starts the connection-establishment timers. T2 controls retransmission, while T3 and T4 simulate acceptance and rejection after independently selected random delays. In CONNECTED, the application starts T5 and prints 'Data Exchange is ongoing...' once per second until T5 expires. The main thread waits on both the Timer Manager notification event and a Ctrl+C event, using bright yellow for messages, bright cyan for states, and bright magenta for timer expiries.

SIMULTANEOUS RESPONSES: T3 and T4 may both be queued when the application drains the shared expiry event. The application processes the first response, performs the corresponding state transition, and stops the peer timer. If the peer expiry is already queued, the application receives it after the state has changed and records it as stale. A deterministic self-test verifies exactly one state transition and one stale response.

9. Concurrency and Synchronization

Concurrency is controlled by assigning each shared resource both an owner and a synchronization rule. The server main thread serializes timer-core access, the client API uses internal locks for communication between application and receiver threads, and console handlers do not perform cleanup concurrently. Sections 9.1 and 9.2 define those rules before Table 7 summarizes them.

9.1 Client API Thread Safety

The client library creates one receiver thread to read ACK and expiry messages independently of application threads. It also stores a Windows INIT_ONCE structure so process-wide initialization runs exactly once even when several application threads call CreateTimer concurrently. That initialization creates three Windows critical sections, which are mutual-exclusion locks used inside the library. Section 9.2 explains how functions obtain the shared client-state pointer before acquiring the required critical section.

The command_mutex lock permits one command and ACK exchange at a time. During that exchange, waiting_request_id stores the request ID expected by the command thread, and last_ack stores the matching ACK copied by the receiver thread. The ack_event object is the

Copyright (c) 2026 Vitaly Drapkin

Windows event that the receiver thread signals after copying that ACK so the waiting command thread can continue. The state_mutex lock protects the connection, receiver lifecycle, request-matching fields, and ACK transfer. The expiry_mutex lock protects queue insertion and removal together with the notification-event state.

WHY LAST_ACK EXISTS: The receiver thread reads all inbound pipe messages, while the thread executing a public command waits on ack_event. When the receiver obtains the ACK whose request ID matches waiting_request_id, it copies that ACK into last_ack while holding state_mutex and then signals ack_event. The command thread acquires state_mutex, reads last_ack, and returns the completed response to the caller.

9.2 Global State Access Rule

Each module groups its mutable process-lifetime data into one structure: vdrServerState_type for the server, vdrClientApiState_type for the client library, and corresponding state structures for the sample and tests. An accessor is a function that returns a pointer to its module's state object. Only vdrServer_GetState(), vdrClientApi_GetState(), SampleGetGlobalState(), or TestGetGlobalState() names the underlying global object directly. Every operational function calls the appropriate accessor once, stores the returned pointer locally, and accesses state members through that pointer.

An accessor defines the ownership boundary but does not lock the returned object. The server's single-thread rule and the client's critical sections still govern every read and update. Immutable static const lookup tables remain outside runtime state because they neither change nor participate in control-flow handoffs.

Table 7. Concurrency ownership and synchronization

Resource	Protection or owner	Why the rule is required
Public command path	command_mutex	Only one request may wait for one matching ACK at a time.
Lazy initialization	command_mutex and INIT_ONCE	Concurrent first calls must create one pipe, one receiver, and one event set.
ACK and pipe state	state_mutex	The receiver and command threads need a protected response and lifecycle handoff.
Expiry FIFO	expiry_mutex	Receiver enqueue and application dequeue must preserve links and event state together.
Timer lists	Server main thread	The timer core contains no internal lock and relies on serialized callers.

Copyright (c) 2026 Vitaly Drapkin

Resource	Protection or owner	Why the rule is required
Shutdown	Manual-reset event	The console handler wakes the main loop instead of performing cleanup concurrently.

The API receiver thread is created and owned by the client library rather than by the named-pipe mechanism. Windows completes overlapped operations and signals events, but application code controls the receiver's lifetime. That distinction matters during cleanup because the API must stop the loop, cancel its read, join the thread, and only then release the handles and queued expiry nodes that the thread could otherwise touch.

10. Error Handling and Defensive Behavior

Validation and error handling occur at each defined interface boundary. Public API functions reject TIMER_ID_INVALID and a zero duration before building a command. The server verifies message metadata before reading payload fields, checks timer ownership during lookup, and returns TIMER_NOT_OWNED when an otherwise valid ID belongs to another pipe.

Failures inside the implementation are translated into stable API outcomes. Allocation and unexpected Windows failures become INTERNAL_ERROR, while connection or communication loss becomes SERVER_NOT_AVAILABLE on the client. On the server, a pipe failure initiates disconnect cleanup so no timer remains associated with a dead owner. The expiry queue also reports an invalid indication or allocation failure instead of silently corrupting its FIFO state.

Rejected work must leave the existing system state usable. Protocol tests send a valid command after malformed input to verify continued communication. Stop and delete tests wait beyond the original deadline to verify that no stale expiry is delivered. Saturating timestamp arithmetic applies the same defensive rule at the numeric boundary by preventing a large duration from wrapping into an immediate expiry.

11. Building and Running

11.1 Prerequisites

The C11 project targets Windows 10 or later and is built with Visual Studio 2022, the MSVC desktop toolchain, and an installed Windows SDK. CMake 3.20 or later generates the solution and registers each test scenario with CTest. CMake configures every project translation unit to compile as C11 without compiler-specific language extensions.

Copyright (c) 2026 Vitaly Drapkin

11.2 Configure and Build

Run the following commands from the repository root. CMake creates an x64 Visual Studio solution in build, and the second command compiles the Debug configuration.

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Debug
```

Under MSVC, every C target compiles with /W4 and /WX. A compiler warning therefore stops the build instead of being accepted as successful output. Non-MSVC configurations use -Wall, -Wextra, and -Werror for the same reason.

11.3 Run

Start build\Debug\vdrapkinTimerManager.exe in the first console and wait until it reports that the named-pipe listener is armed. Then start build\Debug\vdrTimerManagerSampleClient.exe in a second console with no command-line arguments. The sample will print timer expiries, simulated messages, and state transitions as connection cycles repeat. Press Ctrl+C in the sample for orderly application cleanup, then stop the server with Ctrl+C.

11.4 Run Tests

```
ctest --test-dir build -C Debug --output-on-failure
```

The timer-core tests run without a server, while integration, robustness, and sample self-tests require the named-pipe endpoint. The regression workflow starts a temporary hidden server, executes every registered CTest scenario in a separate process where appropriate, and then stops the server. This lifecycle makes cleanup failures visible and prevents one test's client state from leaking into the next.

12. Verification Strategy

The regression suite contains 39 named CTest scenarios arranged in layers. Timer-core tests establish deterministic list and arithmetic behavior before any pipe is involved. Integration tests then exercise the real API and server, robustness tests challenge protocol and capacity boundaries, and the sample self-test resolves simultaneous response expiries. Running scenarios independently isolates the client API's process-lifetime state and associates each documented requirement with a named result.

Copyright (c) 2026 Vitaly Drapkin

Table 8. Automated test suite composition

Layer	Count	Purpose
Timer-core unit	30	These tests establish deterministic list, scheduling, ownership, restart, cleanup, active-list delta preservation, and uint64_t behavior without pipe dependencies.
Client/server integration	9	These tests exercise the real API, named pipe, ACK handoff, expiry event, FIFO queue, cancellation, restart behavior, negative ACK behavior, and queued-expiry event signaling.
Robustness	19	These tests challenge malformed protocol input, stale StartTimer message sizes, maximum-client and wait-handle capacity, disconnect cleanup, ownership failures, immediate-expiry timestamp behavior, and concurrent public API calls.
Sample self-test	1	This test verifies the sample state machine stale-response case when T3 and T4 expiries are both queued.

12.1 Why Negative Tests Matter

A negative test verifies both the returned error code and the state after rejection. It checks that timer lists, ownership, wait duration, connection usability, and event state remain unchanged. The malformed-message tests issue a valid command after rejection to verify connection recovery. The stop and delete prevention tests wait beyond the original deadline to verify that cancellation prevents notification delivery.

12.2 Complete Automated Test Catalog

Table 9 maps each scenario name to the behavior it verifies. The catalog is ordered from timer creation and list invariants through real pipe exchanges, malformed protocol input, exact wait-handle capacity, concurrent API use, and the sample's stale-response rule.

Table 9. Automated test scenario catalog

Layer	Scenario	Verified behavior
Core	CreateTimerCreatesInactiveTimer	Confirms creation, a nonzero ID, and placement in the inactive list.
Core	CreateTimerAllocatesRunningIds	Confirms that successive creations

Copyright (c) 2026 Vitaly Drapkin

Layer	Scenario	Verified behavior
		receive monotonically increasing IDs.
Core	TimerIdWrapSkipsInvalidId	Confirms that counter wrap skips the reserved zero ID.
Core	TimerIdWrapSkipsIdAlreadyInUse	Confirms that wraparound skips IDs that are already allocated.
Core	StartTimerMovesInactiveToActive	Confirms activation and the initial wait duration.
Core	StartTimerRejectsZeroDuration	Confirms that zero duration is rejected without mutating list state.
Core	InvalidTimerIdRejectedByCommands	Confirms that start, stop, and delete reject the reserved zero ID.
Core	WrongOwnerRejectedByCommands	Confirms that one client cannot manipulate another client's timer.
Core	SortedDeltaListUsesScheduledExpiryDeltas	Confirms out-of-order insertion and the resulting scheduled-expiry deltas.
Core	EqualExpiryTimersExpireTogether	Confirms zero-delta peers and one callback for each equal deadline.
Core	ProcessExpiredDoesNotExpireEarly	Confirms that a timer does not expire one millisecond early.
Core	ProcessExpiredMovesTimerBackInactive	Confirms the inactive transition and callback arguments after expiry.
Core	ProcessExpiredWithNullCallbackMovesTimerInactive	Confirms that expiry processing still returns timers inactive when no callback is installed.
Core	ProcessExpiredMultipleStaggeredTimersInOrder	Confirms that staggered expiries are processed in scheduled order.
Core	PartialExpiryLeavesLaterTimersActive	Confirms that removing the due head preserves the later timer and its delay.
Core	ExpiredTimerCanBeRestarted	Confirms that an expired timer can be started again.
Core	RestartActiveTimerWithShorterDuration	Confirms that a shorter restart

Copyright (c) 2026 Vitaly Drapkin

Layer	Scenario	Verified behavior
		moves the timer toward the active head.
Core	RestartActiveTimerWithLongerDuration	Confirms that a longer restart moves the timer behind earlier deadlines.
Core	StopActiveTimerMovesItInactive	Confirms that stop cancels an active schedule and returns the timer inactive.
Core	StopMiddleActiveTimerPreservesFollowingDelta	Confirms that stopping a middle active timer preserves the scheduled expiry of timers behind it.
Core	StopInactiveTimerIsSuccessful	Confirms that stopping an inactive timer remains a successful idempotent operation.
Core	DeleteInactiveTimerRemovesIt	Confirms that deleting an inactive timer frees it and rejects repeated deletion.
Core	DeleteActiveTimerRemovesItAndKeepsOthers	Confirms active deletion without disturbing the timing of neighboring timers.
Core	DeleteMiddleActiveTimerPreservesFollowingDelta	Confirms that deleting a middle active timer preserves the scheduled expiry of following active timers.
Core	GetNextWaitDurationWhenNoActiveTimer	Confirms the empty active-list scheduling result.
Core	RemoveTimersForPipeRemovesOnlyThatOwner	Confirms that disconnect cleanup removes only timers owned by the specified pipe.
Core	RemoveTimersForPipePreservesRemainingActiveDeltas	Confirms that disconnect cleanup recalculates active-list deltas for remaining clients.
Core	ExpiryArithmeticSaturatesAtUint64Max	Confirms that expiry arithmetic saturates instead of wrapping into the past.
Core	NearUint64MaxTimersRemainOrdered	Confirms ordering and delta calculations near UINT64_MAX.

Copyright (c) 2026 Vitaly Drapkin

Layer	Scenario	Verified behavior
Core	DeinitRemovesAllTimers	Confirms that deinitialization releases every timer and resets both lists.
Integration	CreateDeleteRoundTrip	Confirms the basic request, ACK, and delete path through the real pipe.
Integration	StartExpireDeleteRoundTrip	Confirms end-to-end start, expiry delivery, deletion, and an empty queue afterward.
Integration	StopActivePreventsExpiry	Confirms that stopping an active timer prevents a later expiry event.
Integration	StopInactiveRoundTrip	Confirms idempotent stop behavior through the public API and server.
Integration	RestartActiveUsesNewDuration	Confirms that the application receives an expiry indication for the replacement deadline rather than the original deadline.
Integration	DeleteActivePreventsExpiry	Confirms that deleting an active timer prevents a later expiry event.
Integration	MultipleTimersExpireThroughQueue	Confirms that one shared event can represent several queued timer IDs in FIFO order.
Integration	QueuedExpiriesKeepNotificationEventSignaled	Confirms that the notification event remains signaled until every queued expiry has been retrieved.
Integration	NegativeAckStatusesFromUserPerspective	Confirms the public API behavior for negative ACK status values.
Robustness	MalformedMagicRejected	Confirms magic rejection and successful use of the same connection afterward.
Robustness	MalformedVersionRejected	Confirms version rejection and connection recovery.
Robustness	MalformedTypeRejected	Confirms unsupported message-type rejection and connection recovery.

Copyright (c) 2026 Vitaly Drapkin

Layer	Scenario	Verified behavior
Robustness	MalformedTotalSizeRejected	Confirms validation of the exact received byte count.
Robustness	MalformedPayloadSizeRejected	Confirms validation of payload_size against the selected structure.
Robustness	MalformedTruncatedHeaderRejected	Confirms that an incomplete header is rejected before payload interpretation.
Robustness	MalformedOversizedMessageRejected	Confirms that oversized fixed messages are rejected.
Robustness	MalformedPayloadTooSmallRejected	Confirms that a payload smaller than the selected command structure is rejected.
Robustness	MalformedPayloadTooLargeRejected	Confirms that a payload larger than the selected command structure is rejected.
Robustness	MalformedOldStartTimerSizeRejected	Confirms that protocol-version-sized StartTimer messages are rejected after api_start_timestamp_ms was added.
Robustness	MalformedAckSentToServerRejected	Confirms that server-to-client ACK messages are rejected if sent in the client-to-server direction.
Robustness	MalformedExpirySentToServerRejected	Confirms that server-to-client expiry messages are rejected if sent in the client-to-server direction.
Robustness	MaximumClientAndWaitHandleCapacity	Confirms 61 clients, one active timer, 64 wait handles, and rejection of an excess client.
Robustness	MaximumClientCapacityRecoversAfterDisconnect	Confirms that listener capacity recovers after a connected client disconnects.
Robustness	DisconnectCleanupRemovesClientTimers	Confirms that disconnect cleanup removes active and inactive timers owned by the disconnected pipe.

Copyright (c) 2026 Vitaly Drapkin

Layer	Scenario	Verified behavior
Robustness	NegativeAckOwnershipAndRawDuration	Confirms ownership failure and raw-duration validation through negative ACK paths.
Robustness	StartTimerAlreadyExpiredTimestampDeliversImmediateExpiry	Confirms that an already elapsed adjusted duration produces ACK OK followed by asynchronous expiry.
Robustness	RestartActiveTimerAlreadyExpiredTimestampClearsOldSchedule	Confirms that an active restart with an already elapsed timestamp clears the old schedule and delivers one expiry.
Robustness	ConcurrentPublicApiCalls	Confirms eight simultaneous first creates, unique IDs, and complete timer lifecycles.
Sample	SimultaneousResponseExpiries	Confirms one T3 or T4 transition and one stale response when both expiries are queued.

13. Current Limitations and Future Work

The current suite covers every documented user-visible scenario, but it does not inject failure into malloc, event creation, thread creation, or selected Windows API calls. Deterministic tests for disconnects in the middle of an ACK or expiry write would further strengthen cleanup behavior and retry after an initially unavailable server deserves a dedicated lifecycle test.

This version also leaves service hosting, security descriptors, telemetry, benchmarking, and soak testing outside its scope. The server intentionally waits for writes to complete because traceable control flow matters more than throughput in this example. A production design could adopt fully overlapped writes or IOCP after measurements showed that the simpler event-wait model had become the limiting factor.

POSITIONING: This project is a readable RTOS-style example rather than a commercial timer service. Its value comes from making ownership, scheduling, protocol validation, and wait-handle behavior explicit enough to inspect in code and observe in live traces. Production throughput features are intentionally deferred until measurements justify their complexity.

Copyright (c) 2026 Vitaly Drapkin

14. AI-Assisted Development Workflow

This project was intentionally developed using OpenAI Codex as part of the engineering process. The objective was not simply to accelerate implementation, but to evaluate how AI-assisted development can support the design, implementation, review, testing, and documentation of a non-trivial systems software project.

AI-generated output was treated as a proposal rather than as authoritative implementation. Every accepted suggestion was reviewed for correctness, consistency with the documented architecture, coding style, synchronization rules, ownership model, and API contract before incorporation into the project.

Throughout development, architectural decisions, API design, concurrency model, protocol definition, verification strategy, and final technical decisions remained under human control. Codex was used to explore alternative designs, review implementation ideas, suggest test scenarios, improve documentation, and identify opportunities for simplification. Every accepted contribution was reviewed, validated, and, where necessary, revised before becoming part of the project.

Table 10. AI-Assisted development responsibilities

Engineering Activity	Codex Contribution	Engineer Responsibility
System architecture	Discussed architectural alternatives and design tradeoffs	Defined the overall architecture, component responsibilities, and interfaces
Client API	Suggested naming and consistency improvements	Designed the API contract, request/ACK handling, receiver thread, and synchronization
Server implementation	Assisted with implementation ideas, refactoring, and review	Designed the server architecture, scheduling, synchronization, and request processing
Timer core	Suggested implementation of refinements and edge cases	Designed scheduling algorithms, timer lifecycle, and ownership rules
Documentation	Assisted with wording, organization, and editing	Produced the technical content and ensured accuracy
Testing	Suggested additional test cases and corner conditions	Designed the verification strategy and validated correctness
Code review	Identified potential defects, inconsistencies, and simplification opportunities	Evaluated recommendations and made all final engineering decisions

Copyright (c) 2026 Vitaly Drapkin

Appendix A. Project Layout

The repository separates public contracts, timer algorithms, communication endpoints, executable behavior, and verification. Table 11 identifies the file that owns each responsibility so a reader can move from a design concept to its implementation without searching the entire solution.

Table 11. Project file layout

Path	Role in the implementation
include/vdr_timer_manager_protocol.h	Defines shared constants, numeric enum values, protocol structures, and directional message unions.
include/vdr_timer_manager_api.h	Declares the public timer API and documents event-handle ownership.
src/vdr_timer_core.h and src/vdr_timer_core.c	Defines Timer_type and implements active and inactive list algorithms.
src/client/vdr_timer_manager_api.c	Implements the pipe client, receiver thread, ACK handoff, state accessor, and expiry queue.
src/server/vdrapkin_timer_manager_server.c	Implements the console server, wait loop, state accessor, command dispatch, and expiry sending.
samples/sample_client.c	Implements the connection state machine, colored traces, shutdown accessor, and self-test mode.
tests/timer_core_tests.c	Implements 30 deterministic timer-core scenarios and accessor-controlled callback records.
tests/client_api_integration_tests.c	Implements nine API and server integration scenarios.
tests/robustness_tests.c	Implements malformed-protocol, capacity, disconnect-cleanup, timestamp, and concurrency scenarios.
docs/timer_manager_design.md	Records the maintained design decisions and behavior contract.
CMakeLists.txt	Defines build targets, warning policy, and CTest registration.

Copyright (c) 2026 Vitaly Drapkin

Appendix B. Key Constants

The constants in Table 12 bind the protocol identity and Windows wait-set geometry. Changing a protocol constant requires both endpoints to be rebuilt, while changing a capacity constant also requires the wait-handle arithmetic and robustness expectations to be reviewed together.

Table 12. Key constants

Constant	Value	Purpose
PIPE_NAME	\\.\pipe\vdrapkinTimerManager	Names the endpoint that both client and server open.
TIMER_PIPE_MAGIC	0x544D4752	Identifies a message as Timer Manager protocol data.
TIMER_PROTOCOL_VERSION	2	Identifies the protocol revision that carries api_start_timestamp_ms in START_TIMER_REQ.
TIMER_ID_INVALID	0	Reserves a value that can report failure when no timer ID exists.
TIMER_SERVER_MAX_WAIT_HANDLES	64	Matches the Windows WaitForMultipleObjects handle limit.
TIMER_SERVER_MAX_CLIENTS	61	Preserves slots for the listener, waitable timer, and shutdown event.

Appendix C. Design Invariants

A timer exists in exactly one list at a time. Active timers remain ordered by scheduled expiry, expired or stopped timers return to inactive state, and the core invokes the expiry callback only after that transition is complete. Timer ID zero is reserved permanently and is skipped when the running counter wraps.

Ownership is enforced throughout command processing. A command cannot manipulate another client's timer, each valid command produces one generic ACK, and disconnect cleanup deletes every timer owned by that pipe. Each client pipe has at most one outstanding asynchronous read, while each client API process permits at most one command and matching ACK exchange at a time.

Copyright (c) 2026 Vitaly Drapkin

The notification-event state represents whether the expiry queue is empty. The event remains signaled while at least one timer ID is queued and resets only after the application removes the final queued ID. On the server, the wait set never exceeds 64 handles because admission stops at 61 connected clients and reserves one handle for the listener, one for the waitable timer, and one for shutdown.

Copyright (c) 2026 Vitaly Drapkin

Appendix D. Representative Execution Traces

The traces in this appendix were recaptured from the current Debug build after the StartTimer API timestamp change and after the current regression run passed. They are representative excerpts rather than a separate protocol definition. The full captured files are stored under docs\trace_samples in the GitHub-ready repository tree, and the excerpts preserve the current PRINTOUT_TRACE format with file names and line numbers.

D.1 Sample Application State Machine Run

The sample application creates five timers, enters IDLE, starts T1, and then lets timer expiries drive simulated connection-management messages. In this run, T1 expired first, the application sent CONNECTION_REQUEST, T2 caused one retry, and T4 delivered CONNECTION_REJECT. Because T4 won over T3, the application stopped T3 before returning to IDLE. The StartTimer API trace shows api_start_timestamp_ms, which is captured before the request is sent to the server.

```
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:1041] [sample-client][state] initial state=IDLE
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=1
duration_ms=5000 api_start_timestamp_ms=448976534
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T1 expired timer=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:610] [sample-client][message][TX] CONNECTION_REQUEST
attempt=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:295] [sample-client][state] IDLE ->
WAITING_FOR_CONNECTION_ESTABLISHMENT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:616] [sample-client][scenario] T3_accept_ms=24000
T4_reject_ms=14000 expected_response=CONNECTION_REJECT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=5000 api_start_timestamp_ms=448981539
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=3
duration_ms=24000 api_start_timestamp_ms=448981540
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=4
duration_ms=14000 api_start_timestamp_ms=448981541
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T2 expired timer=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:648] [sample-client][message][TX] CONNECTION_REQUEST
attempt=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=4000 api_start_timestamp_ms=448986550
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T2 expired timer=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:648] [sample-client][message][TX] CONNECTION_REQUEST
attempt=3
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
```

Copyright (c) 2026 Vitaly Drapkin

```
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=3000 api_start_timestamp_ms=448990565
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T2 expired timer=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:648] [sample-client][message][TX] CONNECTION_REQUEST
attempt=4
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=5000 api_start_timestamp_ms=448993577
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T4 expired timer=4
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:720] [sample-client][message][RX] CONNECTION_REJECT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:388] [sample-client][timer] stopping T3 timer=3
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:295] [sample-client][state]
WAITING_FOR_CONNECTION_ESTABLISHMENT -> IDLE
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=1
duration_ms=7000 api_start_timestamp_ms=448995592
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T1 expired timer=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:610] [sample-client][message][TX] CONNECTION_REQUEST
attempt=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:295] [sample-client][state] IDLE ->
WAITING_FOR_CONNECTION_ESTABLISHMENT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:616] [sample-client][scenario] T3_accept_ms=19000
T4_reject_ms=5000 expected_response=CONNECTION_REJECT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=4000 api_start_timestamp_ms=449002609
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=3
duration_ms=19000 api_start_timestamp_ms=449002615
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=4
duration_ms=5000 api_start_timestamp_ms=449002617
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T2 expired timer=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:648] [sample-client][message][TX] CONNECTION_REQUEST
attempt=2
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=2
duration_ms=4000 api_start_timestamp_ms=449006665
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T4 expired timer=4
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:720] [sample-client][message][RX] CONNECTION_REJECT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:388] [sample-client][timer] stopping T3 timer=3
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:295] [sample-client][state]
WAITING_FOR_CONNECTION_ESTABLISHMENT -> IDLE
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\client\vdr_timer_manager_api.c:1027] [client-api] StartTimer accepted timer=1
duration_ms=6000 api_start_timestamp_ms=449007653
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:842] [sample-client][timer-expiry] T1 expired timer=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:610] [sample-client][message][TX] CONNECTION_REQUEST
attempt=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:295] [sample-client][state] IDLE ->
```

Copyright (c) 2026 Vitaly Drapkin

```
WAITING_FOR_CONNECTION_ESTABLISHMENT
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\samples\sample_client\sample_client.c:616] [sample-client][scenario] T3_accept_ms=13000
T4_reject_ms=16000 expected_response=CONNECTION_ACCEPT
```

D.2 Server StartTimer Adjustment and Expiry Delivery

The server trace shows how the timestamp is used. When the StartTimer command is dispatched, the server records server_now_ms, calculates elapsed_ms, and schedules the adjusted duration. When the waitable timer later expires, the timer core moves the timer back to the inactive list and the server sends TIMER_EXPIRED_EVT to the owning client pipe.

```
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1000] [server][command] dispatch StartTimer
request=6 timer=1 duration_ms=5000 api_start_timestamp_ms=448976534 server_now_ms=448976534
elapsed_ms=0 adjusted_duration_ms=5000
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1038] [server][command] core StartTimer done
timer=1 requested_duration_ms=5000 adjusted_duration_ms=5000 status=OK(0) immediate_expiry=0
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1169] [server][timer-expiry] begin
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1145] [server][timer-expiry] sending indication
timer=1 pipe=00000000000000A8
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:613] [server][send] begin pipe=00000000000000A8
type=TIMER_EXPIRED_EVT request=0 bytes=24
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:734] [server][expiry-send] sent timer expiry
indication timer=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1154] [server][timer-expiry] indication complete
timer=1
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1179] [server][timer-expiry] rearming waitable
timer after expiry processing
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1181] [server][timer-expiry] complete
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1169] [server][timer-expiry] begin
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:613] [server][send] begin pipe=00000000000000A8
type=TIMER_EXPIRED_EVT request=0 bytes=24
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1179] [server][timer-expiry] rearming waitable
timer after expiry processing
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1181] [server][timer-expiry] complete
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1169] [server][timer-expiry] begin
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:613] [server][send] begin pipe=00000000000000A8
type=TIMER_EXPIRED_EVT request=0 bytes=24
[C:\Users\vdrap\OneDrive\Documents\Timer management\windows-rtos-style-timer-
manager\src\server\vdrapkin_timer_manager_server.c:1179] [server][timer-expiry] rearming waitable
timer after expiry processing
```

Copyright (c) 2026 Vitaly Drapkin

D.3 Automated Regression Result

The regression trace was captured with a fresh server process running. This matters because the integration, robustness, and sample self-test cases communicate through the named pipe and therefore require the server executable to be active.

```
      Start 53: vdrTimerManagerRobustnessTests.MaximumClientCapacityRecoversAfterDisconnect
53/59 Test #53: vdrTimerManagerRobustnessTests.MaximumClientCapacityRecoversAfterDisconnect
.................   Passed    0.79 sec
      Start 54: vdrTimerManagerRobustnessTests.DisconnectCleanupRemovesClientTimers
54/59 Test #54: vdrTimerManagerRobustnessTests.DisconnectCleanupRemovesClientTimers
.........................   Passed    0.39 sec
      Start 55: vdrTimerManagerRobustnessTests.NegativeAckOwnershipAndRawDuration
55/59 Test #55: vdrTimerManagerRobustnessTests.NegativeAckOwnershipAndRawDuration
............................   Passed    0.18 sec
      Start 56:
vdrTimerManagerRobustnessTests.StartTimerAlreadyExpiredTimestampDeliversImmediateExpiry
56/59 Test #56:
vdrTimerManagerRobustnessTests.StartTimerAlreadyExpiredTimestampDeliversImmediateExpiry .....
Passed    0.17 sec
      Start 57:
vdrTimerManagerRobustnessTests.RestartActiveTimerAlreadyExpiredTimestampClearsOldSchedule
57/59 Test #57:
vdrTimerManagerRobustnessTests.RestartActiveTimerAlreadyExpiredTimestampClearsOldSchedule ...
Passed    0.18 sec
      Start 58: vdrTimerManagerRobustnessTests.ConcurrentPublicApiCalls
58/59 Test #58: vdrTimerManagerRobustnessTests.ConcurrentPublicApiCalls
.....................................   Passed    0.15 sec
      Start 59: vdrTimerManagerSampleClient.SimultaneousResponseExpiries
59/59 Test #59: vdrTimerManagerSampleClient.SimultaneousResponseExpiries
.....................................   Passed    0.38 sec

100% tests passed, 0 tests failed out of 59

Total Test time (real) =  13.66 sec
```

Appendix E. StartTimer API Timestamp Revision

This appendix documents the protocol-version-2 change that makes the timer start timestamp belong to the client API call rather than to the later server dispatch point. It supersedes the earlier protocol-version-1 StartTimer structure.

Windows may delay the server thread after an application has already called StartTimer. If the countdown begins only when the server eventually dispatches the pipe command, unrelated operating-system scheduling delay extends the effective timer duration. The revised design records the start timestamp in the client API before the request is sent, allowing the server to subtract the elapsed transport and scheduling delay from the requested duration.

TIMER_PROTOCOL_VERSION is now 2. vdrStartTimerReq_type contains api_start_timestamp_ms, a monotonic millisecond timestamp captured by the client API. Older version-1-sized StartTimer messages are rejected because the total size and payload_size no longer match the protocol-version-2 command structure.

Copyright (c) 2026 Vitaly Drapkin

When the server receives StartTimer, it captures server_now_ms, calculates elapsed_ms from the API timestamp, and then calculates adjusted_duration_ms. If adjusted_duration_ms is greater than zero, the server starts or restarts the timer core using that adjusted duration.

If adjusted_duration_ms is zero, the requested duration has already elapsed before server dispatch. The server uses the existing timer-core StopTimer path to leave the timer inactive, sends ACK OK first, and only after that sends the asynchronous expiry event. This preserves the synchronous command contract while still reporting the elapsed timer to the application.

```
elapsed_ms = server_now_ms - api_start_timestamp_ms
adjusted_duration_ms = duration_ms - elapsed_ms
```

Current traces show the client API timestamp on each StartTimer request through api_start_timestamp_ms. Expiry delivery is shown by the server TIMER_EXPIRED_EVT trace and by the client API line where the application retrieves the expired timer ID.

Copyright (c) 2026 Vitaly Drapkin